

SD Basic - File Handling

Opening files, reading and writing records, locking, and transactions.

This page covers the statements that reach the database: opening a file, reading and writing whole records or single fields, the locking that keeps two sessions out of each other's way, and transactions.

Sequential files — ordinary operating-system files read a line at a time — are in SD Basic - Sequential Files. Select lists are in SD Basic - Select Lists.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Opening

```
open {dict,} file.name to file.variable {readonly}
    then statements
else statements

openpath pathname to file.variable {readonly}
    then statements
else statements
```

`open` looks the name up in the account's VOC. `openpath` takes an operating system path and bypasses the VOC entirely.

The `else` branch is where a missing file arrives, and it is not optional. A program that omits it does not compile. Never leave it empty:

```
open 'CUSTOMERS' to f.cust else
    stop 'Cannot open CUSTOMERS'
end
```

`dict` **opens the dictionary rather than the data portion** — `open 'dict', 'CUSTOMERS' to f.dict else ...`.

`close file.variable` releases it. A file left open is closed when the program ends, so `close` matters mainly in a long-running program that opens files in a loop.

Reading

```
read variable from file.variable, record.id then ... else ...
readv variable from file.variable, record.id, field then ... else ...
matread matrix from file.variable, record.id then ... else ...
```

<code>read</code>	the whole record into one dynamic array
<code>readv</code>	a single field
<code>matread</code>	the record spread across a dimensioned matrix, one field per element

For a record holding `alpha` and `beta|gamma`:

RESULT	
<code>read got from f, 'R1'</code>	<code>then</code> branch; <code>dcount(got, @fm)</code> is <code>2</code>
<code>read got from f, 'NOSUCH'</code>	<code>else</code> branch
<code>readv fv from f, 'R1', 2</code>	<code>beta\ gamma</code>

A missing record is the `else` branch, not an error. And `status()` after one reads **3006**, which is how you tell "no such record" from a real failure such as a permission problem.

`matread` **puts everything past the end of the matrix into element zero.** A three-field record read into `dim mm(2)` gives `mm(1) = f1`, `mm(2) = f2` and `mm(0) = f3`. Nothing is lost and nothing is reported.

`inmat()` **does not tell you it happened** — it reads `0` after that `matread`, not `3`. It reports the element count after `matparse`, not after `matread`. If a record may have more fields than your matrix, `read` it and use `dcount(rec, @fm)`, or size the matrix from the dictionary.

Writing

```
write variable to file.variable, record.id
writev variable to file.variable, record.id, field
matwrite matrix to file.variable, record.id
```

Writing creates the record if it does not exist and replaces it if it does. There is no "insert only" form — read first if that distinction matters, and hold a lock between the read and the write or two sessions will both find it absent.

`delete file.variable, record.id` removes a record. After `delete f, 'R2'`, reading `R2` takes the `else` branch.

`clearfile file.variable` empties the file but leaves it in place.

Locking

SD locks a **record**, not a row or a page, and the lock belongs to the session.

```
readu variable from file.variable, record.id {locked ...} then ... else ...
readvu variable from file.variable, record.id, field ...
matreadu matrix from file.variable, record.id ...
recordlocku file.variable, record.id
recordlockl file.variable, record.id
release {file.variable {, record.id}}
```

<code>readu</code>	read and take an update lock
<code>readl</code>	read and take a shared read lock
<code>recordlocku</code>	take an update lock without reading
<code>release</code>	give locks back

`write` releases the lock it finds. `release` with no arguments releases everything the session holds.

```
recordlocked(file.variable, record.id)
```

reports the state. After `readu`, `recordlocked(f, 'R1')` is **2**; an unlocked record reads **0**; after `release f, 'R1'` it is **0** again.

The `locked` clause is the only way not to wait. Without it, a `readu` against a record another session holds **blocks until that session lets go**. With it, control goes to the `locked` branch immediately:

```
readu rec from f, id locked
  print 'Someone else is editing that record.'
  return
end then
...
```

```
end else
    ...
end
```

A program without a `locked` clause is a program that can hang a user indefinitely with no message. **Any interactive read-for-update should have one.**

`filelock` and `fileunlock` lock a whole file, and `lock` / `unlock` take one of 64 numbered general-purpose semaphores — used to serialise something that is not a record at all, such as a report run.

The rest of this is its own page. What the other session sees, what every `recordlocked()` code means, how to find out **who** holds a lock, and how long a wait actually is, are all in SD Basic - Locks and Transactions, which covers what two sessions running at once see of each other.

Transactions

```
begin transaction
    ...
    commit
    ...
    rollback
end transaction
```

Everything written between `begin transaction` and `commit` either all lands or none of it does. `rollback` discards the lot.

Locks taken inside a transaction are held until it ends, whichever way it ends. That is what makes the transaction safe and also what makes a long one expensive — keep the span short, and never wait for user input inside one.

And inside a transaction you must already hold the lock on every record you write. The same `write` that works outside one fails inside it with *"Error 3023 (o/s 0) writing record (Possible full disk?)"*, which is a lock error wearing a disk error's message. SD Basic - Locks and Transactions has the detail, along with what `commit`, `rollback` and simply reaching `end transaction` each do.

Asking about a file

```
fileinfo(file.variable, key)
```

On a newly created dynamic file:

KEY	MEANING	RESULT
0	is this variable an open file?	1
1	the VOC name it was opened by	ZZWORK
2	the path on disk	/cygdrive/c/ProgramData/SD/user_accounts/don/ZZWORK
3	file type	3
5	modulus	
6	minimum modulus	1
7	group size	
11	current load percentage	
13	does the file have alternate key indexes?	
1001	read-only?	
1003	size in bytes, excluding indexes	

File types: 1 SH, 3 **DH — an ordinary dynamic file**, 4 directory, 5 sequential.

The PATH comes back in POSIX form, not as a Windows path — /cygdrive/c/ProgramData/..., not C:\ProgramData\... **Handing that string to a Windows program does not work** — Windows reads it as a drive-relative path and either fails silently or reports that the parent directory does not exist. This is not a theoretical caution: it is what stopped the full-screen editors working the first time they were built for this port.

And the conversion function is not available to an ordinary program. The kernel can convert such a path, and kernel() is an internal-only intrinsic — a kernel(...) call in a user account does not compile, and the compiler's complaint is "Matrix KERNEL is not referenced in a DIM statement", reported at the last line of the program. See SD Basic - System and Environment. **So a path a Windows program is going to see should come from your own configuration, not from fileinfo().**

There is no record-count key. Key 6 is the *minimum modulus* and reads 1 on a small file whatever it contains — 1 with two records present and 1 again after clearfile. To count records, select the file and read selectinfo(list, 3), or count in a readnext loop.

dir(pathname) lists an operating-system directory as a dynamic array — name, size and modification time per entry.

Reaching another file's data

```
trans({dict} file.name, record.id, field, action)
rtrans({dict} file.name, record.id, field, action)
xlate({dict} file.name, record.id, field, action)
```

All three fetch one field of one record from a named file, opening it if needed. `trans` and `xlate` are the same function under two names; `rtrans` takes an already-open file variable instead of a name.

The *action* says what to do when the record is missing:

x	return the null string
v	return the null string and print an error
c	return the record id itself
n	return the record id itself

A *field* of `0` returns the record id.

These open a file on every call unless it is already open. In a loop over many records that is a real cost — `open` the file once and use `rtrans`, or read it directly.

Creating and configuring

```
create.file path {directory | dynamic}
create file.variable then ... else ...
configure.file file.variable, key, value
```

Most applications create files with the `create.file` **command** rather than the statement — from BASIC, `execute 'create.file ...'` is the usual route, and it is what the examples on this page use.

`set.trigger` attaches a program that runs on every write or delete to a file. A trigger runs inside the caller's transaction, so anything it does is part of the same all-or-nothing unit.

What is not here

Nothing in the file-handling group has been removed from this port, but two things about it changed:

VFS	the virtual file system layer has been removed from the C entirely . <code>fileinfo()</code> never reports a VFS type, and the type code is gone
The data tree is private	<code>C:\ProgramData\SD</code> is protected by an access-control list. A file created by SD is reachable through SD, and not by an ordinary Windows user poking at the directory

See also

SD Basic - Select Lists · SD Basic - Dynamic Arrays · SD Basic - Sequential Files · SD Basic - Alternate Key Indexes.